2026-06-30-vpn-aware-proxy-extension-design

Design Spec — Helix Proxy: VPN-Aware Dynamic Routing Extension

Revision: 4 **Last modified:** 2026-07-01T02:00:00Z **Status:** Active — P1 spike gaps (G1–G4) resolved with captured evidence; P4 Squid/Dante templates parse-verified; §9 stale Dante-SIGHUP marker reconciled to the G3 spike outcome (Rev 4); implementation in progress (companion plan P2–P12) **Authority:** Inherits the Helix Constitution submodule (`constitution/Constitution.md`) per §11.4.35. **Sources (research, captured this session):** `docs/research/mvp/findings/A_vpn_multitunnel_orchestration.md`, `…/B_squid_dante_dynamic_routing.md`, `…/C_antibluff_live_evidence.md`, `…/D_bleeding_edge_features.md`, `…/E_existing_tests_antibluff_audit.md`, plus the upstream research scaffold `docs/research/mvp/proxy-vpn-extension/`.

---

1. Problem & Goal

The shipped research material (“VPN-Aware Proxy Extension”) proposes routing proxy traffic to hosts reachable **only via VPN**, with **zero interruption** when tunnels toggle, plus **dynamic per-target routing** and **horizontal scale**. The existing `helix_proxy` System already routes **all** traffic through **one** VPN (or none) via compose profiles, but it cannot:

- route **specific targets** through **specific VPN profiles**,
- run **multiple concurrent tunnels**,
- return an **immediate graceful 503** when a target’s tunnel is down (today a down tunnel = dropped/hung traffic, no clean signal),

- prove any of the above with **captured evidence** (the functional test suite is largely a §11.4 bluff — see §15).

Goal: Add a **dynamic control-plane** that delivers per-target VPN-profile routing, live tunnel state, graceful degradation, multi-tunnel resilience, observability, DNS privacy, zero-trust security, and a real admin/API — as an **additive** layer over the existing OpenVPN/Squid/Dante stack (no removal of existing capability without operator confirmation, §11.4.122) — and prove every capability with **rock-solid data-plane evidence** and **no bluff** (§11.4).

2. Existing System (ground truth)

Rootless-Podman + compose orchestrated (`init`/`start`/`stop`/`restart`/`status` shell scripts sourcing `lib/container-runtime.sh`). Services:

- **proxy-vpn** — `dperson/openvpn-client` VPN gateway (NET_ADMIN, /dev/net/tun, routes for 192.168/16, 10/8, 172.16/12, kill-firewall on, ping healthcheck).
- **proxy-squid** — `ubuntu/squid` HTTP/HTTPS caching proxy (:53128).
- **proxy-dante** — `vimagick/dante` SOCKS5 (:51080).
- **proxy-admin** — `traefik/whoami` **placeholder** (:58080).
- **cache-invalidator**, **vpn-monitor** — alpine sidecars.

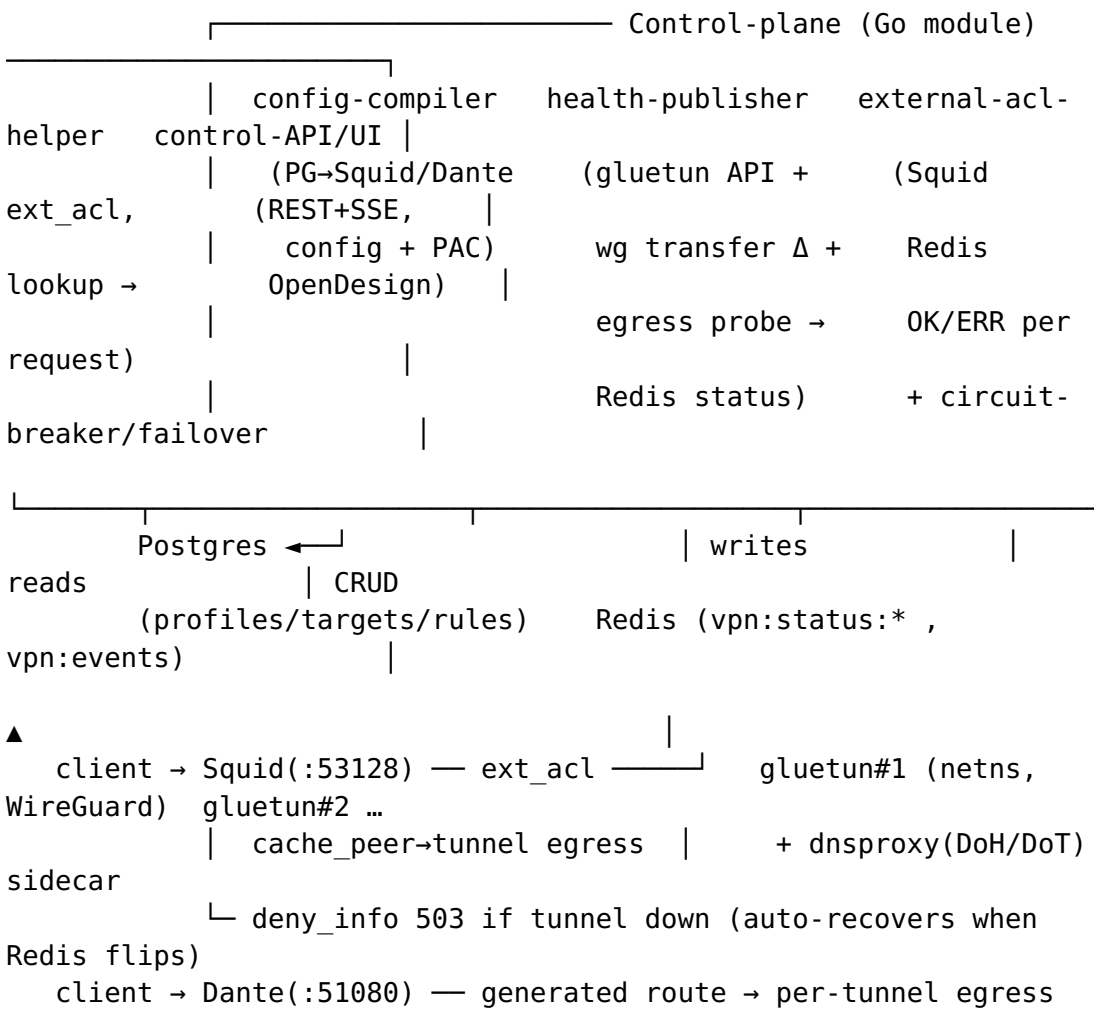
Three compose profiles: `vpn` (proxies share VPN netns via `network_mode: service:proxy-vpn`), `host-vpn` (`network_mode: host`), `no-vpn`. Configs (`config/squid/squid.conf`, `config/dante/sockd.conf`) are init-generated forward-proxy configs with **no per-target upstream selection**. **No** Postgres/Redis, **no** Go code, **no** dynamic routing.

3. Research-doc mapping (what we keep, what we change)

Research doc proposed	Decision
Gin `httputil.ReverseProxy`, Host→target lookup, proxy reimplemented in Go	**Reject** as the data path. Keep Squid/Dante as the proxy/cache; the Go control-plane is a **config-compiler + health-publisher + ACL helper out of the byte path** (Stream B).
Postgres `vpn_profiles`/`target_hosts`/`proxy_rules`	**Keep + extend** (tunnel failover tiers, auth, audit) — §6.

Research doc proposed	Decision
Redis `vpn:status:<profile>` + `vpn:events`	**Keep** as the live data-plane state bus — §7.
VPN Manager applies WireGuard/OpenVPN, publishes up/down	**Replace the stub** with gluetun-per-profile + a real health-publisher that reads **data-plane** signals (Stream A/C).
K8s DaemonSet + hostNetwork	**Defer** (no K8s in this System today); design stays K8s-portable but ships compose-first.
Hardcoded creds in compose, `your-registry/`	**Reject** — Podman secrets, §12.

4. Architecture — control-plane out of the byte path



Principle (Stream B/C): the control-plane never sits in the request byte path. Squid keeps proxying + caching; a tiny Go `external_acl_type` helper consults Redis per request. This preserves Squid's mature caching **and** gives dynamic per-target VPN selection + graceful 503 with **zero reconfigure**.

Components (each one clear purpose, testable in isolation)

1. **vpn-health-publisher** — per tunnel: poll gluetun control API (`/v1/vpn/status`, `/v1/publicip/ip`) **and** `wg show <if> transfer` byte-delta **and** a live egress probe; write `vpn:status:<profile>` (up/down, handshake age, rx/tx, egress_ip, checked_at) + publish `vpn:events`. Health = **data-plane** fact, never "configured" (Stream A/C).
2. **config-compiler** — read Postgres → render: Squid generated include (per-tunnel `cache_peer`, `cache_peer_access`, `never_direct allow all`, `deny_info 503:ERR_TUNNEL_DOWN`), Dante route config, and a PAC file. Apply via Redis (per-request) for routing/up-down; `reconfigure`/`SIGHUP` only for **structural** changes (new tunnel/peer) — never for up/down (Stream B).
3. **external-acl-helper** — Squid `external_acl_type` Go binary: per request read Redis `{target→tunnel, up?}` → `OK tag=<tunnel>` (allow that peer) or `ERR` (→ `deny_info 503`). Embeds the **circuit-breaker + tunnel tier-failover** decision (gobreaker).
4. **control-API + admin UI** — REST CRUD (profiles/targets/rules), live status (SSE), Prometheus metrics, `FindProxyForURL` PAC endpoint. `templ`+`htmx`+SSE, **OpenDesign tokens (§11.4.162)**, light+dark, proven by **host-rendered pixel proof (§11.4.170)**. Replaces `traefik/whoami`.

5. VPN layer (Stream A)

- **gluetun (`qmcgaw/gluetun`, MIT) — one container per `vpn_profile` = one netns = one `vpn:status:<profile>` key.** Maps 1:1 onto the existing `network_mode: service:<vpn>` pattern (§11.4.74 reuse, not reinvent).
- **Kernel WireGuard preferred** (throughput/CPU/audit-surface); OpenVPN as compatibility type. Rootless-WireGuard risk → fall back to gluetun **userspace wireguard-go** (spike, §20).
- Existing `dperson/openvpn-client` → **retained as a `legacy` profile, marked deprecated** (NOT removed — §11.4.122 operator-confirmation required to drop).
- gluetun kill-switch drops packets on down; the **graceful L7 503 we build** (helper), because the kill-switch alone yields hangs, not a clean signal.

6. Data model (Postgres — extended from research doc)

`vpn_profiles(id, name, type[wireguard|openvpn], config jsonb, enabled, …)`, `target_hosts(id, public_alias, private_ip, port, protocol, vpn_profile_id, health_check, enabled, …)`, `proxy_rules(id, priority, match_host, match_path, target_host_id, enabled, …)` — **plus**: `target_tunnel_tiers(target_id, vpn_profile_id, tier)` (ordered failover list, §11 feature ①), `proxy_users(id, username, secret_ref, …)` (auth, §12), `audit_log(id, ts, actor, action, detail)` (§12). Full DDL + migrations + ER diagram ship under `sql/` + `docs/`.

7. Redis contract

- `vpn:status:<profile>` — JSON `{state, last_handshake, rx, tx, egress_ip, checked_at}` (TTL ⇒ stale = treated **down**, fail-closed).
- `vpn:events` — pub/sub `{profile_id, state}` for instant helper/UI updates.
- `route:<target>` — resolved `{tunnel, tier, breaker_state}` (compiler-written).

8. Squid integration (Stream B)

Fixed base `squid.conf` + generated include: per-tunnel `cache_peer <gluetun-host> parent <port> 0 no-query name=tun_<profile>; external_acl_type vpn_route ttl=0 concurrency=… %>{Host} <helper>; acl tun_up external vpn_route; cache_peer_access tun_<p> allow …; never_direct allow all; deny_info 503:ERR_TUNNEL_DOWN tunnel_down.` **Verify Squid major version** — `tcp_outgoing_address` dstdomain form **removed in v8** (spike, §20). 503-on-down + auto-recovery come from the helper flipping OK/ERR on the Redis state — **no reconfigure**.

9. Dante integration (Stream B)

Generated `route { ... via <per-tunnel upstream> }` / `external.rotation route`; reload via `SIGHUP`. **Dante has no external-helper hook**, so SOCKS dynamism = config rewrite + SIGHUP. **SIGHUP preserves an active SOCKS session — RESOLVED FACT** (G3 spike PASS: 20/20 chunks, `curl` exit 0, `/proc/net/tcp` ESTABLISHED proof across the reload; §20 G3 + `docs/research/mvp/findings/F_spikes_G1-G4.md`). The narrower

route-change-mid-session behaviour (does an in-flight session keep its OLD path on a reload that changes its route?) stays `UNCONFIRMED:` and is owed to P9 (§20 G3).

10. Error handling & failure modes (fail-closed, no leak)

- Tunnel down → helper `ERR` → branded **503** (Squid PID unchanged — proven).
- Redis down / stale key → **fail-closed 503** (never fall through to a leak).
- Circuit-breaker open for a target → **failover** to next up tier, else 503.
- Control-plane crash → Squid keeps serving last good config (degrade, not die).
- gluetun kill-switch → **zero** packets on the real uplink while down (proven by `tcpdump`, §14).

11. Bleeding-edge features (Stream D — all MVP-now)

① **per-target circuit breakers + tunnel tier-failover** (`sony/gobreaker/v2`). ② **DoH/DoT per-tunnel DNS + leak prevention** (`AdguardTeam/dnsproxy` sidecar). ③ **observability** — `boynux/squid-exporter` + Prometheus + Grafana + OTel spans in the Go control-plane. ④ **zero-trust** — Squid per-user auth + audit + Podman secrets + in-netns kill-switch + mTLS on the control-API. ⑤ **dynamic PAC + health/geo routing + hot-reload + split-tunnel** (Go `FindProxyForURL`). ⑥ **real admin UI + REST API** (templ+htmx+SSE, OpenDesign §11.4.162, pixel proof §11.4.170). ⑦ **IPv6 dual-stack + keep-alive/pool tuning**. ⑧ **cache analytics + Store-ID URL normalization**.

Deferred — honest §11.4.112 boundaries: HTTP/3/MASQUE forward-proxy (Squid cannot; early adoption) · streaming/range caching in Squid (structurally limited — `range_offset_limit` removed in v8; needs a different engine) · eBPF/Cilium-Hubble (**AVOID** — needs kernel privileges, breaks the rootless mandate; in-process OTel/metrics give equivalent insight).

12. Security & secrets

VPN creds, proxy-auth secrets, mTLS keys via **Podman secrets / file refs** — **never** plaintext in git (§11.4.10); `.env.example` documents refs only. Per-user Squid auth, audit log, in-netns kill-switch, mTLS on control-API.

13. Anti-bluff test strategy (the heart — §11.4 / §11.4.69 / §11.4.107)

Only data-plane signals are evidence (Stream C). For every capability, the required captured-evidence probe:

Capability	PASS evidence (captured)	Don't-be-fooled gotcha
http_forward / https_connect	real `curl -x` 200 + body	—
cache_hit	`X-Cache: HIT` + `access.log TCP_HIT` + `store.log SWAPOUT` + 2nd-req latency drop	`?`-URLs uncacheable; a header alone is forgeable
vpn_real_egress	egress IP via proxy `==` tunnel exit && `!=` host IP, **+ `wg transfer` Δ**	200 OK ≠ routing; egress-IP cache → pair with counter
graceful_503	tunnel down → 503 + branded body + **PID unchanged** → up → 200	a 503 from a crashed proxy isn't graceful
no_leak/killswitch	drop tunnel → **zero** target packets on real uplink (`tcpdump`) + DNS only via intended resolver	leak-testing while up proves nothing
vpn_reconnect	kill tunnel → status down→up auto	—
cache_invalidation	populate → invalidate → eviction observed	—
circuit_breaker/ failover	force target failures → breaker opens → fail to next tunnel	—
socks5_egress	`curl --socks5-hostname` exit IP	plain `socks5://` leaks DNS — use `socks5h`
admin UI	host-rendered pixel proof (§11.4.170) + real REST CRUD	value/token-equality tests are NOT proof

All test types (§11.4.169): unit (Go `-short`, mocks OK), integration (real PG/Redis/Squid/gluetun booted on-demand via the **containers submodule**, §11.4.76), e2e, full-automation (re-runnable, no manual step, §11.4.98), security (auth/leak/mTLS),

DDoS/load (`k6`/`vegeta`), stress, chaos (**toxiproxy** — rootless-safe; `pumba` needs rootful — caveat captured), concurrency, race (`-race`), memory (soak), benchmark. **Challenges** (one per capability, real evidence) + **HelixQA banks** + **paired §1.1 mutation** per test + **§11.4.115 RED-on-broken-artifact** + **§11.4.135 standing regression guard**.

14. Live-evidence harness

Shared `tests/lib/evidence.sh` (`ab_pass_with_evidence`, `ab_skip_with_reason`, data-plane probes: `assert_egress_ip`, `assert_cache_hit`, `assert_graceful_503`, `assert_no_leak`, `wg_transfer_delta`). bats-core/shellspec orchestration; evidence artifacts under `qa-results/<run-id>/` + curated proof under `docs/qa/<run-id>/` (§11.4.83). Window-scoped, project-prefixed recordings (§11.4.154/§11.4.155) for any UI/terminal capture.

15. Existing-test remediation (Stream E — fix the named bluffs)

1. **False VPN-routing PASS** (`final-verify.sh:74`, `verify-proxy.sh:49`, `comprehensive-test.sh:400`): `host_ip == proxy_ip` PASSes with **no VPN**. Replace with `egress != host_ip && == tunnel_exit` + `wg` Δ; reproduce the bluff first (§11.4.115/§11.4.138 bluff-audit + permanent guard).
2. **Cache-timing bluff** (`comprehensive-test.sh:456`): assert `X-Cache`/`TCP_HIT` on a cacheable URL, not timing on random bytes.
3. **Concurrent FAIL-bluff** (`comprehensive-test.sh:633`): capture `%{http_code}`, not job exit status.
4. **Presence-only `run-tests.sh`** (`test_vpn:273`, `test_service_startup:311`): convert PASS-by-default to real probes or honest §11.4.3 SKIP. The solid **governance** tests/mutation are preserved as-is.

16. Governance, docs & submodules

- **Carriers:** add project `QWEN.md` + `GEMINI.md` (§11.4.157 lockstep); ensure the §11.4 anti-bluff covenant + the verbatim “tests pass but features unusable” anchor + the **full-docs-sync directive** are present in `CLAUDE.md`/`AGENTS.md`/`CONSTITUTION.md`/`QWEN.md`/`GEMINI.md`. The canonical text lives in the constitution submodule (§11.4.35) — carriers cite/inherit.
- **Required submodules (§11.4.27/§11.4.74):** `vasic-digital/containers` (§11.4.76 orchestration + §11.4.173 build), `HelixDevelopment/HelixQA`, `vasic-digital/challenges`, `vasic-digital/docs_chain` (§11.4.106). Added via SSH + `install_upstreams` (§11.4.36), recursive.

- **Docs always-in-sync (§11.4.12/.44/.60/.65/.106):** ARCHITECTURE/CACHE/VPN docs updated; new SQL/ diagrams/graphs; HTML+PDF (+DOCX where mandated) exports; bound by **docs_chain** so docs↔DB↔diagrams never drift. README + USER_GUIDE updated for the dynamic mode.

17. Build & deploy (§11.4.76 / §11.4.173)

New compose profile **`dynamic`** brings up postgres + redis + control-plane + gluetun(s) + squid(+helper) + dante. Existing `vpn`/`host-vpn`/`no-vpn` preserved. `init` generates configs + provisions Podman secrets; `start`/ `status`/`stop` learn the `dynamic` profile. Go control-plane builds via the **containers submodule** on the remote build host (§11.4.173), artifacts brought back. Orchestration through the containers submodule, not ad-hoc commands.

18. Module / file layout

```
control-plane/                  # new Go module
  go.mod  cmd/{healthd,acl-helper,compiler,api}/  internal/
{vpn,routing,store,redis,breaker,api,pac,otel}/
sql/                            # DDL + migrations
config/squid/squid.conf.base    # + generated includes
config/dante/                   # + generated routes
tests/lib/evidence.sh           # data-plane evidence helpers
tests/{unit,integration,e2e,security,load,chaos,...}/
challenges/scripts/<capability>_challenge.sh
docs/{ARCHITECTURE,CACHE,VPN,DYNAMIC_ROUTING}.md + diagrams/ + qa/
.docs_chain/contexts/*.yaml     # docs_chain wiring
```

19. Out of scope / deferred / structural impossibilities

- HTTP/3 forward-proxy, Squid range-caching, eBPF — §11 deferred list (honest §11.4.112). K8s DaemonSet — design stays portable, compose ships first.

20. Open questions / honest gaps — RESOLVED by P1 spikes (captured evidence)

All four gaps are resolved as FACT with captured evidence (§11.4.123); full detail + evidence paths in `docs/research/mvp/findings/F_spikes_G1-G4.md`.

- **G1 — RESOLVED (nuanced):** kernel-WireGuard *interface creation* succeeds rootless inside a container with `--cap-add NET_ADMIN` + `--device /dev/net/tun`; gluetun auto-selects kernelspace, with userspace `wireguard-go` as the portability fallback. **Decision:** keep gluetun auto-select; required container grants are `--cap-add NET_ADMIN` + `--device /dev/net/tun`. Full rootless kernel-WG *operation* (handshake+traffic) stays `UNCONFIRMED:` → proven live in P10.
- **G2 — RESOLVED:** `ubuntu/squid:latest` = **Squid 6.13** (NOT v8); the full §8 directive set (`cache_peer` / `external_acl_type` / `cache_peer_access` / `never_direct` / `deny_info`) parses clean (`squid -k parse` exit 0). **Decision:** pin the Squid base by **digest** or a specific `6.13-<ubuntu>_<channel>` tag — docker.io/ubuntu/squid has **NO bare `6.13` tag** (P4-templates finding; published tags are `6.13-25.04_*` / `latest` / `edge`, and `:latest` is verified = 6.13); the config-compiler emits **`%>ha{Host}`** (6.13 deprecates `%>{Host}`); the v8 `tcp_outgoing_address`-dstdomain concern is moot (design routes via `cache_peer` + `external_acl`, not `tcp_outgoing_address`). Squid template `squid -k parse` exit 0 on real 6.13 (P4-templates). Dante config is assembled by **concatenation** (Dante has no `include` directive — P4-templates finding), base preserved + per-tunnel `route{}` blocks appended.
- **G3 — RESOLVED (spike PASS):** Dante `SIGHUP` reloads config and **preserves an active SOCKS session** (20/20 chunks, curl exit 0; `/proc/net/tcp` ESTABLISHED proof). **Decision:** §9 config-rewrite + SIGHUP dynamism is safe for live sessions. P9 still owes a broader live test (concurrent / repeated SIGHUP + route-change-mid-session → `UNCONFIRMED:` whether an in-flight session keeps its old path).
- **G4 — RESOLVED:** gluetun pinned **`v3.40` (=v3.40.4)**; control API `:8000` (`/v1/vpn/status`, `/v1/publicip/ip`) answers 200 (publicip empty without real egress — the live `vpn_real_egress` evidence source, §13); internal healthcheck on `127.0.0.1:9999`. **Issue #3060 confirmed live** (routes open-by-default on v3.40.x). **Decision:** pin v3.40; **any upgrade past v3.40 MUST add control-server auth** (apikey/role) — a pinned-version constraint + upgrade-checklist item aligning §11 ④ zero-trust.

21. Phasing (full detail in the companion plan)

P0 governance+submodules+scaffold → P1 spikes (G1–G4) → P2 data model+stores → P3 health-publisher → P4 config-compiler+Squid/Dante integration → P5 external-acl-helper+503+failover → P6 control-API+admin UI → P7 DNS/ observability/ security → P8 fix existing bluffs → P9 full test matrix+Challenges+HelixQA → P10 live validation+evidence → P11 docs sync+exports → P12 review+release (no-force §11.4.113, prefixed tag §11.4.151). Each phase: TDD reproduce-first, all test types, paired mutation, captured evidence, independent code review (§11.4.142/ .125/.134), zero bluff.